Computer Architecture & Concurrency – COMP0008
Lecture 1: Introduction

Kevin Bryson
k.bryson@ucl.ac.uk
Darwin Building Room 632.

Course Overview

- Part I: Computer Architecture
  - Overview of key components of computer architecture
  - MIPS32 Architecture and Assembly Language
  - Block diagram of how a MIPS processor “works”
  - Instruction-level parallism (pipelining, super-scalar architecture)
  - Thread-level parallism (multicore, memory coherence/consistency)
  - Small bit about operating systems (processes, threads, running executables, multithreading, low-level concurrency primitives created from assembly instructions).

- Part II: Java concurrent programming
  - Concurrency Abstraction ... understanding concurrent systems.
  - Interference, visibility issues, safety/liveness balance, deadlock,...
  - Java concurrency mechanisms (locks, mutexes, semaphores, monitor design, ...)
  - The Java concurrency package
  - Ways of coordinating threads so they work together.

Course Organization

- The course has asynchronous lectures / videos and activities at the start of the week.
- There are Q&A sessions on Wed. / Fri. to see if there are any questions or issues with the activities (and also to demo how things work if needed).

- Three courseworks during Term 1 / 2 each worth 10% :
  - Electran online system testing fundamental architecture topics and MIPS assembly language programming.
  - Moodle quizzes on Java concurrency ...
  - Multi-threaded Java programming exercise ...
- Then a comprehensive Term 3 “Open Book Assessment” which is worth the other 70% of the mark.

- *Open office* on Wed. 8am to 9am for individual questions ...

Textbooks … Computer Architecture

- "**Computer Organization and Design (4th Edition)**",
  by Patterson and Hennessy, 2009, ISBN 978-0-12-374493-7
  (The MIPS edition ... not the more recent ARM or RISC-V editions)

Key textbook about the MIPS processor architecture (written by the Professors who designed the chip).

- **"Digital Design and Computer Architecture"**
  by David Money Harris and Sarah L. Harris, 2013, ISBN: 0123944244

MIPS architecture, MIPS assembly, (we will **not** be focusing on the electronics/logic gates aspects that you may have done last year)

***** Both physical and online copies in Library *****

Textbooks … Concurrency

- **“Java concurrency in practice”**
  by Brian Goetz. Addison-Wesley, 2006.

Advanced text giving you an expert understand of Java concurrency and key design principles.

- **“Concurrency: state models & Java programs”**
  by J. Magee and J. Kramer. John Wiley & Sons Ltd, 2006.

Covers a lot of the key ‘concurrency concepts’ of the course with demos of each concept but also a lot of additional more formal stuff that we will not cover (involving Finite State Machine Analysis).

***** Both physical and online copies in Library *****

OK … to begin … well … at the end …

Part II: Overview and motivation behind concurrent programming.

This is going to be a rather informal look at the concept of concurrency just to get a feeling for the idea and some motivation behind it.

In later lectures we will make the concepts **much** more formal.

My morning routine program ...

1. Enter Kitchen
2. Look into Fridge

3. If (milk < 0.1 litres) {
    1. Go to shop.
    2. Buy milk.
    3. Bring home.
    4. Put in Fridge

}

4. Eat porridge ...

“Why is there too much milk in my fridge?”

1) I find no milk in fridge – “My wife is hopeless!”

2) I go out and buy some milk

5) I put the milk in the fridge

3) My wife finds no milk in fridge – “My husband is hopeless!”

4) They go out and buy some milk

6) “Where did all this milk come from?!” “And you teach concurrency? I feel sorry for your students”

Can you think of a mechanism to avoid this undesirable behaviour?

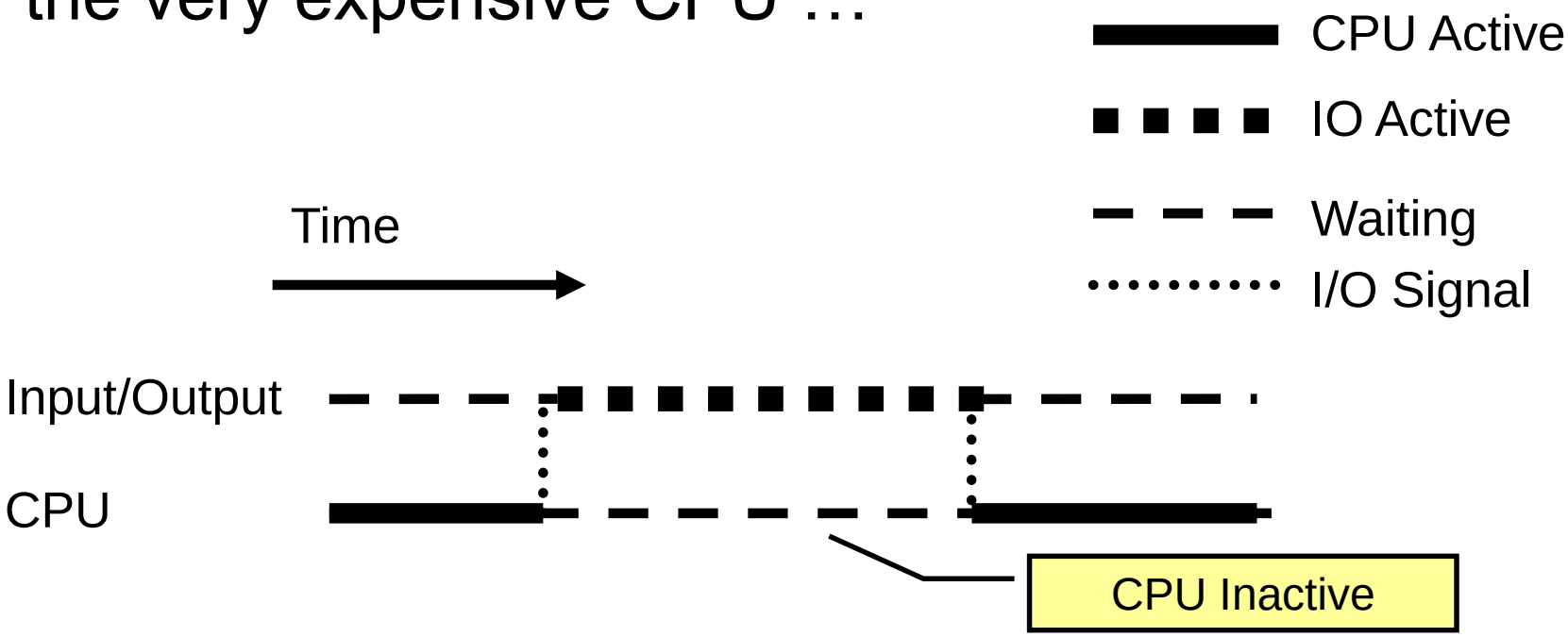
So what is a concurrent program?

- A sequential program consists of instructions that are *totally ordered.* The instructions are carried out in a precise sequence (well that's what you think is happening !) and the **process is deterministic**.
- In reality the order of instructions can be changed by the compiler/processor ('out of order execution') ... but the compiler/processor **guarantees** the same deterministic results from running the program.

- A concurrent program is just a set of ordinary sequential programs executed at the 'same time' (i.e. concurrently).
- We will more formally introduce this idea of concurrency a bit later – but first an overview of some key terminology and the motivation behind it.

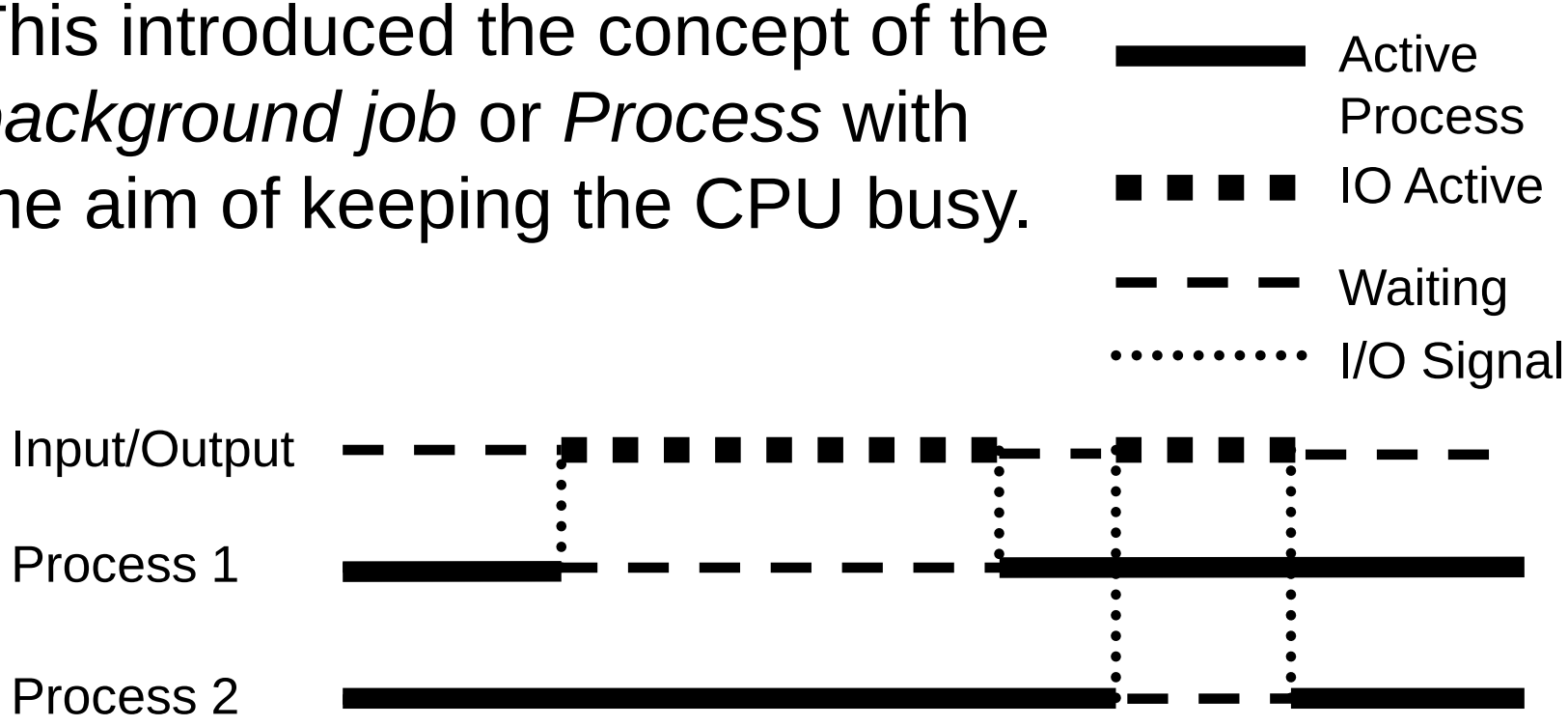
Historically people first developed the ideas of concurrency within operating systems …

- Initial batch machines relied on an operator loading one program onto the machine at a time.
- This lead to very inefficient use of the very expensive CPU …



First *Time Sharing* Operating Systems

- One of the first time-sharing or multi-tasking operating systems was CTSS (Compatible Time Sharing System) developed at MIT in 1961.
- This introduced the concept of the *background job* or *Process* with the aim of keeping the CPU busy.

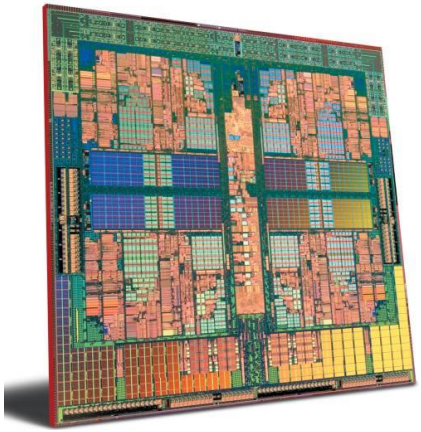


Interleaved Concurrency and True Parallelism

- These early multi-tasking systems are examples of *interleaved concurrency* since there is only one *execution engine or processor* (so there cannot be true parallel processing). Usually the single processor has to switch (rapidly) between the different processes.

- The milk code is an example of *true parallelism* since there were two execution engines (in the average student house many more execution engines trying to use a kitchen ...)

Why “mashup” concurrent programming with the topic of computer architecture ?



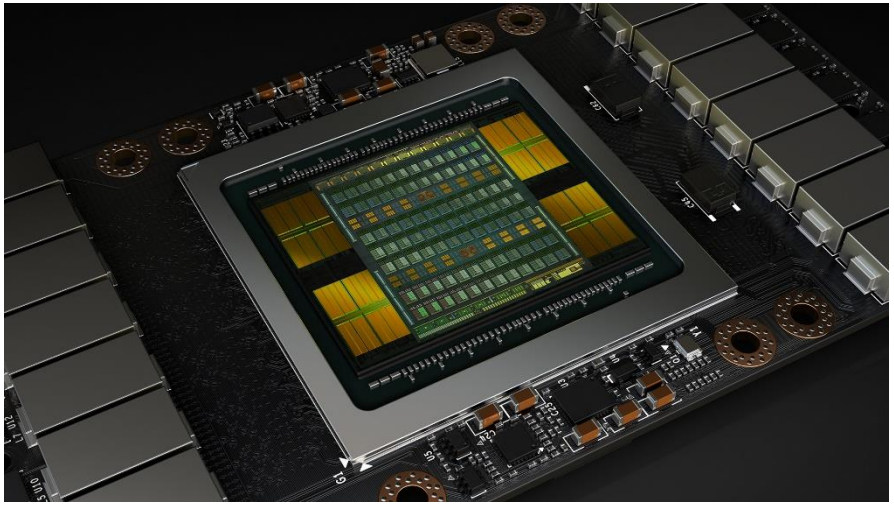
Four “execution engines” ...

Why did CPU architectures switch to multicore around the 2000s ?

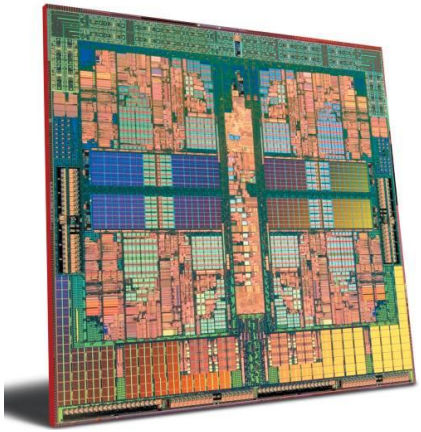
Taking this one stage further ...

What type of processor is this ?

How many execution engines?



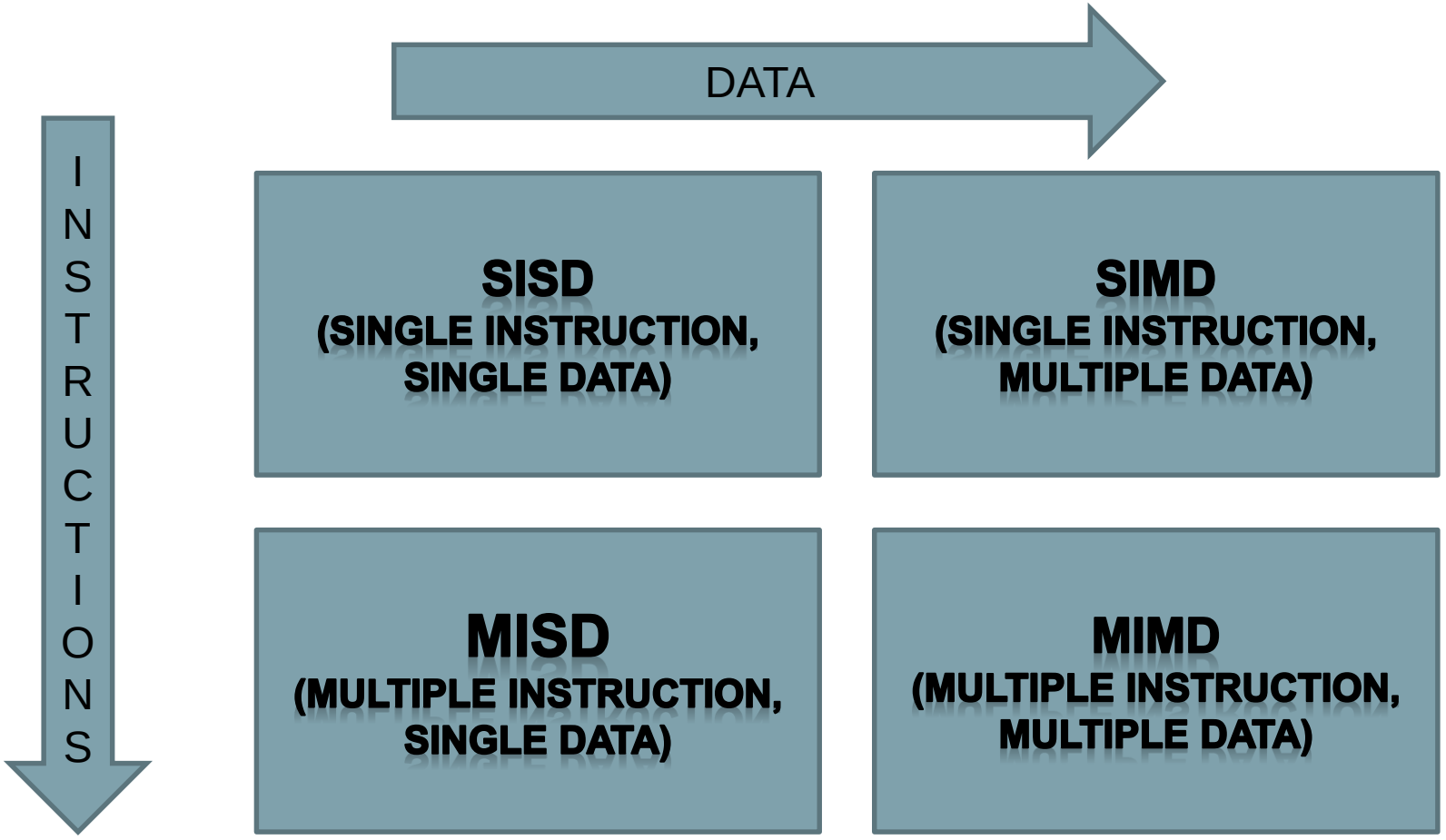
Understanding concurrency and computer architecture is vital in this new world of multi-core CPUs and GPUs



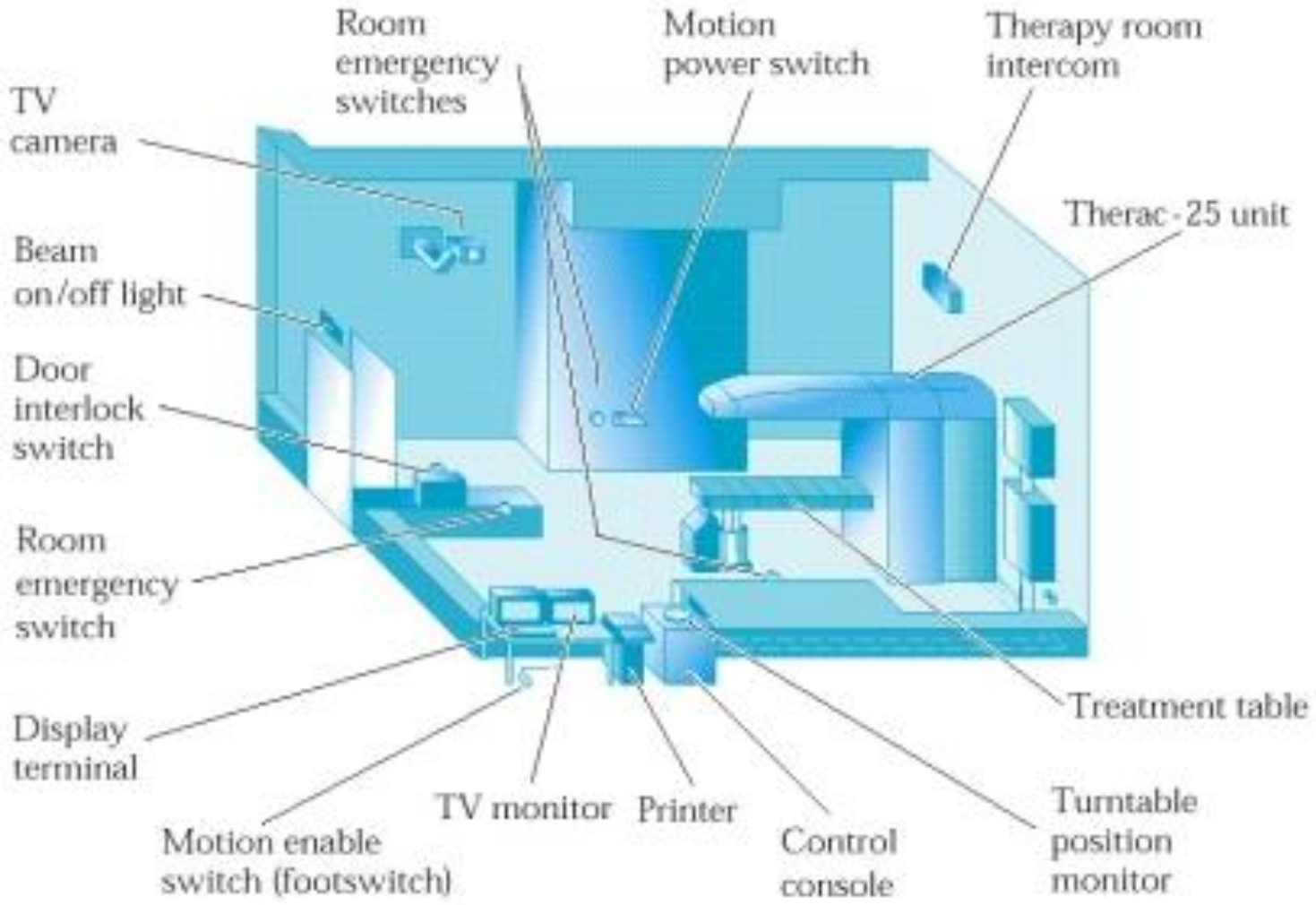
- Both designing and programming this hardware requires concurrency and hardware to be considered.
- NVIDIA Tesla K80 GPU board has 4992 cores and can deliver over 8.74 Tflops of peak floating point performance.
- Programmed using a special version of C called CUDA.

- How do we characterize these processors?

Flynn's Taxonomy of Machines (1972)



Example of an embedded computer controller handling lots of concurrent sensing/tasks: Therac-25 Radiation Therapy Machine



At least 5 people died from massive radiation overdoses between 1985 and 1987 while undergoing radiation therapy on these machines

- Who killed these people ?

- A software engineer (amongst others including engineers, managers, QA testers, mechanical engineers, etc.)

- How ?

- They didn't understand concurrent programming (amongst other problems including software reuse …)

Therac-25 Radiation Therapy Software

- Fine during formal testing …
- Went into production … seemed to work well.
- The operators become more and more efficient at using the interfaces … as they became efficient they pushed buttons in the **correct sequence ... but faster!**
- And the software could not handle it because of concurrenty bugs called **“race conditions”** … resulting in a metal target not being put into place … resulting in massive overdoses of radiation.
- But the software company showed that the software was working fine … look … must be mechanical failure ?
- It took two years and five people’s lives before the company fully investigated the software … and found concurrency bugs which had not shown up in previous systems due to hardware interlocks being present.

But maybe they were not very good software engineers? Or maybe concurrency is difficult …

```
... some other class ...
public Holder holder;
public void initialize() {
        holder = new Holder(42);
}
...

public class Holder {
  private int n;

  public Holder (int n) {
     this.n = n;
  }

  public void my_test() {
    if (n != n) {
      throw new ExceptionError("This statement is false!");
    }
  }
}
```

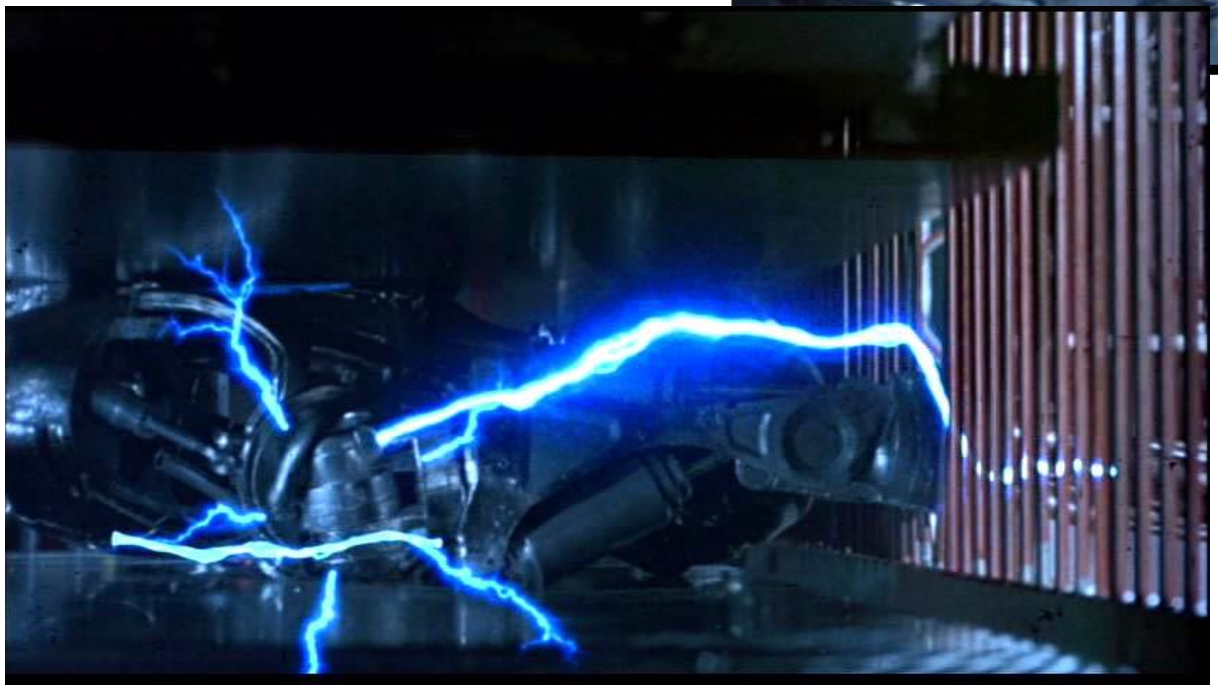
Let's have a vote –

who thinks the message could be printed ?

This is the other reason why we need fundamentally to understand what the hardware is doing.

I don’t believe these strange things can happen ...

Seems like fantasy to me ...



Sarah has an effective way of terminating this rogue thread of execution ... but can we do it in Java code ?!

The Terminator class … does it work ?

```
public class Terminator extends Thread {
    private boolean crush = false;

    public void run() {

        while (true) {
            if (crush) {
                System.out.println(
                    "I see crush == true ... I'll be back!");
                break;
            }
            System.out.print("I am just doing what I was
programmed to do ... \n");
        }
    }

    public void crush() {
        crush = true;
    }
}
```

Surely this should work since we are just reading/writing a Boolean value ? Let's see if it works ...

```
public class Main {

    public static void main(String[] args)
                            throws InterruptedException {

        Terminator robot  = new Terminator();
        robot.start();

        Thread.sleep(5000);
        System.out.println("Crushing the robot ...\n");
        robot.crush();

    }
}
```

So what is Concurrency?

- It is a key abstraction in Computer Science
- Relevant to hardware design, operating systems, multiprocessors, distributed computing, programming and design.

- *Concurrency is the ability to run multiple activities simultaneously or in parallel* (and reason correctly about these systems)

Why is Concurrent Programming required?

- **Performance gain from multiprocessing hardware**
  - **Parallelism**
- Increased application throughput
  - A blocking I/O call only blocks one thread
- Increased application responsiveness
  - High priority thread for user requests
- More appropriate structure
  - For programs which control multiple activities and handle multiple events
- Embedded systems for driving hardware (like Therac-25)
  - Equipment might naturally consist of multiple sensors/activators.
- Distributed systems

So to end with … the start …

Part I: Overview and motivation behind computer architecture and hardware

Again this is an informal introduction to computer architecture ...

In later lectures we will look at the detailed internal structure of MIPS processors and how interaction with memory ***actually works***.

To be continued ... *I'll be back !*